

SISTEMA DISTRIBUÍDO PARA GESTÃO DE TCC: COREOGRAFIA DE EVENTOS SOBRE ZEROMQ E APOIO DE IA

2020032560 - BRUNO STANLLEY DE JESUS

2018016327 - LARISSA FAUSTINO FRANKLIN

2020028058 - PATRICKI HERNANDES DA COSTA

2020028352 - PATRICKY ALEXANDRE DA COSTA SILVA

PROJETO FINAL

Prof. Rafael Frinhani



INSTITUTO DE
MATEMÁTICA E
COMPUTAÇÃO

UNIFEI - Itajubá



Sistema Distribuído para Gestão de TCC: Coreografia de Eventos sobre ZeroMQ e Apoio de IA

Resumo. A gestão de Trabalhos de Conclusão de Curso (TCC) reúne agentes autônomos (discentes, orientadores, banca e coordenação) que cooperam de forma concorrente e assíncrona, o que a caracteriza como um problema de sistemas distribuídos. Este trabalho projeta e implementa um protótipo funcional de um sistema distribuído para a gestão de TCC, adotando o *middleware* ZeroMQ sob o paradigma *brokerless* e coordenação por coreografia, no qual oito serviços autônomos reagem a eventos sem *broker* nem orquestrador central. A solução combina uma interface web em React, uma borda *Backend-for-Frontend* e serviços em Python, persistência em MySQL como fonte da verdade e um serviço de apoio por inteligência artificial com provedor de modelo de linguagem plugável. A avaliação demonstrou a viabilidade técnica: a cadeia de eventos opera de ponta a ponta com latência da ordem de milissegundos e sustenta carga sem perdas.

Palavras-chave: Sistemas Distribuídos; ZeroMQ; Coreografia; Mensageria *Brokerless*; Gestão de TCC.

1 Introdução

O Trabalho de Conclusão de Curso (TCC) constitui uma das etapas mais relevantes da formação em nível superior, na qual o discente articula, de maneira autônoma e sob orientação docente, os conhecimentos construídos ao longo da graduação. Por desenvolver-se durante um ou mais semestres e por envolver múltiplos agentes e artefatos, o TCC pode ser compreendido como um projeto que demanda planejamento, execução e controle sistemáticos, à semelhança das práticas consolidadas de gerenciamento de projetos (Dorneles et al., 2023). A crescente digitalização dos serviços acadêmicos torna natural — e cada vez mais necessária — a adoção de ferramentas computacionais capazes de apoiar a organização, o acompanhamento e a comunicação inerentes a esse processo.

Apesar de sua importância, a gestão de TCC ainda é marcada por significativa complexidade administrativa e burocrática. Entre as principais dificuldades destacam-se o acompanhamento do progresso dos trabalhos; a entrega e o controle de versões de documentos (proposta de trabalho, versões parciais e versão final); o gerenciamento da composição das bancas avaliadoras; e o agendamento das defesas. A esse conjunto de atividades operacionais soma-se a necessidade de disponibilizar, ao coordenador de TCC e a órgãos colegiados como o Núcleo Docente Estruturante (NDE) e o colegiado do curso, relatórios gerenciais que ofereçam um panorama da situação dos trabalhos — por exemplo, a quantidade de defesas realizadas no prazo, a distribuição dos trabalhos por área de pesquisa e a identificação de trabalhos com potencial de publicação. Atualmente,

contudo, a comunicação entre os envolvidos ocorre de forma descentralizada e sem um padrão para o registro e a disponibilização das revisões feitas pelos avaliadores; é comum, inclusive, que um mesmo avaliador analise simultaneamente trabalhos de diferentes alunos utilizando formatos e meios distintos. A ausência de uma ferramenta integrada de apoio torna o processo mais complexo, lento e custoso, exigindo tempo e esforço consideráveis de todos os participantes. Soma-se a esse esforço o tempo despendido pelos orientadores com correções básicas e recorrentes, que poderiam ser apoiadas por recursos automatizados.

O problema situa-se no domínio dos serviços digitais de gestão acadêmica, caracterizado pela interação de múltiplos atores autônomos — discentes, orientadores, coordenador de TCC, membros das bancas avaliadoras e órgãos colegiados — que atuam de maneira concorrente, em diferentes locais e momentos. Esses agentes não compartilham memória nem um espaço de execução comum: cooperam exclusivamente por meio da troca de informações, como documentos, pareceres, notificações e agendas, ao longo de toda a vida do trabalho. Essa natureza intrinsecamente distribuída, somada à exigência de concorrência (várias orientações e avaliações em paralelo) e de assincronia (os agentes raramente estão ativos no mesmo instante), caracteriza o cenário como um problema típico de sistemas distribuídos.

A motivação para a solução proposta reside, portanto, na oportunidade de substituir um processo fragmentado por uma plataforma integrada que centralize a informação e coordene a comunicação entre os agentes. Espera-se, com isso, ampliar a transparência e a rastreabilidade do andamento dos trabalhos, padronizar o registro das revisões, reduzir a carga operacional do coordenador e dos docentes e oferecer aos órgãos colegiados uma visão gerencial consolidada. Iniciativas anteriores evidenciam a pertinência dessa abordagem: sistemas especializados de gestão de TCC já foram propostos, contemplando funcionalidades de comunicação, acompanhamento de progresso, organização de etapas e bibliotecas de trabalhos aprovados (Dorneles et al., 2023). A proposta inspira-se, ademais, em plataformas consagradas de gestão do ciclo de submissão e avaliação de trabalhos acadêmicos, como o JEMS (Sociedade Brasileira de Computação, 2026), adaptando seus conceitos ao contexto de TCC. O presente projeto avança nessa direção ao tratar o problema, de modo explícito, sob a ótica dos sistemas distribuídos e da comunicação entre processos.

A adoção de uma abordagem baseada em sistemas distribuídos justifica-se pela própria natureza do problema: como os agentes são processos independentes, sem memória compartilhada, a única forma de interação é a troca explícita de men-

sagens (*message passing*) (Frinhani, 2026a), o que favorece o desacoplamento, a escalabilidade e a comunicação assíncrona requerida pelo domínio. Para o *middleware* de comunicação, optou-se pelo ZeroMQ sob o paradigma *brokerless*, no qual as mensagens trafegam diretamente entre os pares, sem um servidor central (*broker*) que se torne ponto único de falha e gargalo (ZeroMQ, 2026a); a coordenação entre os serviços adota a **coreografia**, sem orquestrador central. A escolha encontra respaldo na experiência do CERN, que avaliou o ZeroMQ como a melhor opção frente a alternativas como CORBA, Ice e Thrift (Treat, 2014; Lauener & Sliwinski, 2017), e no fato de ser uma tecnologia livre, alinhada a soluções sustentáveis.

Diante do exposto, o presente trabalho tem como objetivo geral projetar e implementar um protótipo funcional de um sistema distribuído para a gestão de Trabalhos de Conclusão de Curso, contemplando os núcleos de *frontend* e *backend* e empregando o *middleware* ZeroMQ sob o paradigma de mensageria *brokerless* com coordenação por **coreografia**, e incorporando um serviço de apoio por **inteligência artificial** ao acompanhamento dos trabalhos. Busca-se, com a solução, integrar e coordenar a comunicação entre os diferentes agentes do processo de TCC e aplicar, de forma prática, conceitos fundamentais de sistemas distribuídos — tais como tempo, coordenação e transações —, demonstrando a viabilidade técnica da proposta por meio da comunicação efetiva entre processos distribuídos.

2 Fundamentação Teórica

Esta seção apresenta os fundamentos teóricos e tecnológicos que embasam a solução proposta. Parte-se da classificação dos paradigmas de comunicação distribuída, situando o projeto no paradigma orientado a mensagens, e detalha-se o *middleware* adotado o ZeroMQ, seus padrões de comunicação, trabalhos relacionados e a justificativa técnica das escolhas.

2.1 Paradigmas de comunicação em sistemas distribuídos

Como processos distribuídos não compartilham memória, toda interação ocorre pela troca explícita de mensagens. A literatura organiza os mecanismos de comunicação em três paradigmas: a *Inter-Process Communication* (IPC), baseada em *sockets* e troca explícita de mensagens; a *Remote Invocation* (RPC e RMI), na qual a troca de mensagens é ocultada por chamadas de procedimentos ou métodos; e a *Message-Oriented Communication*, que estrutura a troca de mensagens em alto nível, tipicamente de forma assíncrona e indireta (Frinhani, 2026a). A comunicação pode ainda ser síncrona bloqueante e de alto acoplamento temporal ou assíncrona não bloqueante, orientada a eventos e de menor acoplamento, ao custo de maior complexidade de ordenação e consistência (Frinhani, 2026a). Dada a autonomia dos agentes do processo de TCC e o fato de raramente estarem ativos no mesmo instante, este trabalho adota o paradigma orientado a mensagens, de natureza assíncrona.

2.2 Middleware orientado a mensagens e desacoplamento

Um *middleware* orientado a mensagens (MOM) oferece comunicação assíncrona de alto nível, na qual os processos trocam mensagens enfileiradas, promovendo o desacoplamento entre emissor e receptor (Frinhani, 2026a). Esse desacoplamento manifesta-se em duas dimensões: a *espacial* (ou referencial), em que o emissor não precisa conhecer a identidade do receptor; e a *temporal*, em que emissor e receptor não precisam estar ativos simultaneamente. Em arquiteturas baseadas em *broker*, como RabbitMQ e Kafka, um componente intermediário medeia e **persiste** as mensagens, garantindo naturalmente ambas as dimensões porém à custa de um componente central, que pode tornar-se gargalo e ponto único de falha (Frinhani, 2026a; Treat, 2014).

É essencial precisar como o desacoplamento se comporta em uma arquitetura *brokerless*, característica central da tecnologia adotada. Sem um intermediário que armazene mensagens, o desacoplamento **temporal não é provido pelo transporte**: se o receptor está inativo, a mensagem não é persistida e pode ser perdida; obtê-lo exige um mecanismo explícito de persistência na camada de aplicação. Já o desacoplamento **espacial é apenas parcial**: embora, no padrão *publish-subscribe*, o emissor não conheça seus assinantes, os pares ainda estabelecem conexões a *endpoints* explícitos, de modo que não há transparência referencial automática. Não é correto, portanto, afirmar independência total entre emissor e receptor em uma solução *brokerless* desprovida de persistência.

Importa, ainda, distinguir dois eixos de descentralização que este trabalho combina, mas que não se confundem. O primeiro é a **infraestrutura de comunicação**: uma solução *brokerless* (como o ZeroMQ) dispensa um *broker* central e os pares trocam mensagens diretamente, em oposição às soluções baseadas em *broker*. O segundo é a **coordenação do processo**: na **coreografia** não há um orquestrador central que dirija o fluxo cada serviço reage de forma autônoma aos eventos de seu interesse e o comportamento global emerge dessas reações, em oposição à **orquestração**, na qual um coordenador central conduz a execução (Frinhani, 2026b). Os eixos são independentes: é possível, por exemplo, ter coreografia sobre um *broker* (o Kafka é um *broker* e, ainda assim, viabiliza arquiteturas coreografadas). A solução proposta combina deliberadamente as duas formas de descentralização: comunicação *brokerless* no transporte e coordenação por coreografia no processo. Em processos de longa duração, como o do TCC, essa coordenação por eventos corresponde ao padrão SAGA: em lugar de uma transação distribuída com confirmação atômica em duas fases (*two-phase commit*), o fluxo é decomposto em uma sequência de transações locais, cada uma com sua compensação em caso de falha (Frinhani, 2026d).

2.3 ZeroMQ: características técnicas

O ZeroMQ é uma biblioteca de mensageria assíncrona de alto desempenho e não um servidor de mensagens, incorporada ao próprio processo da aplicação (ZeroMQ, 2026a; Treat, 2014). Por ser *brokerless*, mantém uma fila de mensagens **por conexão** dentro de cada *socket*, fazendo com que as mensagens trafeguem diretamente entre os pares e dispensando um interme-

diário central (ZeroMQ, 2026b).

Seus *sockets* não são *sockets* convencionais (BSD), mas uma abstração sobre uma fila de mensagens assíncrona; por isso são frequentemente descritos como *smart sockets*. Eles tratam de forma transparente o estabelecimento e o encerramento de conexões, a **reconexão automática**, o enquadramento de mensagens (entregues de forma atômica, nunca parcial), a entrega ordenada, as conexões muitos-para-muitos e o balanceamento de carga entre pares (ZeroMQ, 2026b; Treat, 2014). A entrada e saída é assíncrona: uma *thread* de E/S em segundo plano transmite e enfileira as mensagens enquanto a aplicação prossegue seu processamento (Frinhani, 2026a). A mesma API atende a múltiplos transportes TCP (entre máquinas), *inproc* (entre *threads*), IPC (entre processos) e PGM (*multicast*) (Frinhani, 2026a; ZeroMQ, 2026a).

Em desempenho, o ZeroMQ pode superar o uso direto de *sockets* TCP por meio do agrupamento (*batching*) de mensagens, da redução de travessias da pilha de rede e da desativação do algoritmo de Nagle (Treat, 2014). Para mensagens grandes, suporta o envio com **cópia-zero** (*zero-copy*): em vez de copiar o conteúdo entre os *buffers* de usuário e de *kernel*, a posse do *buffer* de dados é transferida diretamente à camada de transporte, reduzindo o uso de CPU e a largura de banda de memória durante a transmissão (Hintjens, 2013).

Quanto às garantias, cada *socket* possui um limite de fila denominado *high-water mark*; ao atingi-lo, e conforme o tipo de *socket*, o ZeroMQ **bloqueia ou descarta** mensagens (ZeroMQ, 2026b). A entrega é de melhor esforço (*best-effort*): garantem-se a atomicidade e a ordenação, mas não a entrega efetiva nem a semântica *exactly-once*; no *publish-subscribe*, mensagens sem assinantes são descartadas (Treat, 2014; ZeroMQ, 2026b). Tais propriedades constituem limitações reais de confiabilidade e tolerância a falhas, que devem ser mitigadas na camada de aplicação por confirmações (*acknowledgements*), identificadores de mensagem para idempotência e estado durável.

2.4 Padrões de comunicação e protocolo de mensagens

O ZeroMQ implementa padrões de mensageria como pares de tipos de *socket* (ZeroMQ, 2026b). No padrão *request-reply*, os *sockets* REQ/REP impõem alternância estrita entre envio e recebimento; sua variante assíncrona DEALER/ROUTER elimina essa restrição, permitindo que o ROUTER enderece pares específicos por identidade e que o DEALER distribua requisições em *round-robin* adequada a servidores que atendem muitos clientes concorrentes. O padrão *publish-subscribe* (PUB/SUB) realiza distribuição um-para-muitos (*fan-out*) com filtragem por tópico, na qual o publicador nunca bloqueia e descarta mensagens destinadas a assinantes ausentes. Já o padrão *pipeline* (PUSH/PULL) distribui tarefas em paralelo (*round-robin*) e coleta resultados (*fair-queuing*), próprio para o processamento por *workers*.

A solução proposta adota a **coreografia** como modelo de coordenação, tendo o *publish-subscribe* como espinha dorsal: cada serviço publica seus eventos e assina os que consome, formando uma **malha direta de pares** um canal dedicado por tipo de serviço, em vez de um canal único com decisão centralizada, o que favorece a escalabilidade, sem orquestrador nem *broker* central. Dois padrões complementam o mo-

delo: DEALER/ROUTER, para consultas e comandos síncronos do cliente, e PUSH/PULL, para a geração distribuída de relatórios. As mensagens são serializadas em **JSON**, formato textual, autodescritivo e neutro em relação à linguagem (Frinhani, 2026a), com esquema explícito contendo evento, operação, identificador, *timestamp* e *payload*. O uso de JSON viabiliza a interoperabilidade entre o *frontend* (JavaScript/React) e o *backend* (Python) e supera, em robustez de engenharia, protocolos textuais *ad hoc*.

2.5 Trabalhos relacionados

A maturidade do ZeroMQ em sistemas de larga escala é evidenciada pelo projeto RDA3 do CERN, no qual a biblioteca substituiu o CORBA como camada de transporte do *framework* que controla todos os aceleradores de partículas da instituição; o projeto adotou os *sockets* DEALER/ROUTER para comunicação assíncrona e uma única *thread* despachante, relatando ganhos expressivos de escalabilidade (Lauener & Sliwinski, 2017). Essa adequação é coerente com o estudo comparativo conduzido pelo CERN, que avaliou o ZeroMQ como a melhor opção frente a alternativas como CORBA, Ice e Thrift (Treat, 2014). No domínio específico deste trabalho, foi proposto um sistema integrado de gestão de TCC com arquitetura REST e notificações assíncronas via *broker* RabbitMQ (Dorneles et al., 2023); a presente proposta atua no mesmo domínio, porém diferencia-se por adotar mensageria *brokerless* e coordenação por **coreografia**. O JEMS (*Journal and Event Management System*) da SBC (Sociedade Brasileira de Computação, 2026) gerencia o ciclo de submissão e avaliação de trabalhos científicos; a solução proposta adota conceitos análogos, aplicáveis à gestão de TCC: a submissão de versões, a **atribuição de revisores** (orientador e banca), os **pareceres em formulário padronizado** — que endereçam a ausência de padronização das revisões — e o **ciclo de resposta**, no qual o discente atende às observações antes de avançar. A integração com sistemas institucionais já existentes, como o SIGAA, é considerada uma **evolução futura**: o sistema proposto atuaria como camada de gestão complementar ao processo administrativo atual, a ser estudada com profundidade em outro momento.

No tocante à confiabilidade, o Guia do ZeroMQ descreve padrões de *request-reply* confiáveis, com destaque para o *Mailbox Protocol* (MDP), que introduz um *broker* de serviço responsável por rotear requisições a *workers* registrados, com *heartbeating* e gerência de *workers* (Hintjens, 2013). O MDP representa a resposta do ecossistema ZeroMQ à necessidade de garantias mais fortes de entrega e de gerenciamento de serviços; este trabalho reconhece-o como caminho para tais garantias, mas mantém deliberadamente um transporte *brokerless*, recuperando a durabilidade por meio da camada de persistência e assumindo de forma explícita esse *trade-off*.

2.6 Linguagens, bibliotecas e frameworks adotados

A solução é implementada de forma heterogênea: os **serviços** (*peers*) e a borda **BFF** (*Backend-for-Frontend*) em **Python**, por meio do *binding* **PyZMQ**; e o *frontend* web em **React/JavaScript** (*Vite*), que se comunica com a borda por **HTTP/JSON**. A comunicação entre os dois ambientes é mediada por **JSON**,

que atua como contrato neutro de representação. A persistência é provida pelo **MySQL**, que mantém o estado durável dos TCCs, versões, pareceres e agendas, constituindo a **fonte da verdade** do sistema; é essa camada que recupera o desacoplamento temporal ausente no transporte *brokerless*, permitindo que um par inativo reconcilie o estado posteriormente. O sistema conta, ainda, com um **Serviço de IA** que reage a eventos relevantes, monta um *prompt* e consulta um **Provedor de LLM** externo acessível por uma interface **plugável** (no protótipo, a API do Google Gemini; Ollama como alternativa local), retornando análises textuais ao fluxo.

A escolha do ZeroMQ sob o paradigma *brokerless* justifica-se tecnicamente por quatro fatores: (i) o alto desempenho e a baixa latência da comunicação direta entre pares, reforçados por *batching* e *zero-copy*; (ii) a eliminação do *broker* central como gargalo e ponto único de falha; (iii) o conjunto de padrões de comunicação que se ajustam diretamente às interações do sistema (*request-reply*, *publish-subscribe* e *pipeline*); e (iv) o fato de ser uma tecnologia livre e sem restrições de uso, alinhada à concepção de soluções sustentáveis. Alternativas foram consideradas: um *middleware* baseado em *broker* (como RabbitMQ ou Kafka) ofereceria persistência e desacoplamento temporal nativos, ao custo de um componente central e de maior latência; já uma abordagem de invocação remota (REST ou gRPC) favoreceria a simplicidade da requisição-resposta, mas não expressaria com naturalidade a coreografia orientada a eventos nem o caráter *brokerless* que se deseja demonstrar. Optou-se pelo ZeroMQ por equilibrar desempenho, descentralização e aderência aos conceitos da disciplina. O principal *trade-off* garantias de entrega mais fracas e ausência de persistência nativa é mitigado por mecanismos de aplicação: estado durável em **MySQL**, identificadores de mensagem para idempotência, confirmações e *heartbeats*. A viabilidade dessas escolhas em ambientes críticos e de larga escala encontra respaldo na experiência do CERN (Lauener & Sliwinski, 2017; Treat, 2014).

3 Metodologia

A metodologia descreve o processo adotado pelo grupo para analisar o problema, modelar a solução e implementar o protótipo. Esta seção inicia pela análise do problema; as etapas de modelagem e de implementação são apresentadas nas subseções seguintes.

3.1 Análise do Problema

Esta subseção examina o problema sob a perspectiva de engenharia, identificando os requisitos, os componentes e os cenários de uso que orientam a modelagem da solução.

Descrição do problema. A gestão de TCC reúne atividades atualmente conduzidas de forma fragmentada e sem integração. Sob o ponto de vista analítico, o problema decompõe-se em: (i) o acompanhamento do progresso e dos estados de cada trabalho; (ii) o controle de versões e o compartilhamento de documentos, no qual coexistem proposta, versões parciais e versão final, exigindo identificar de forma inequívoca a versão vigente; (iii) a coordenação da comunicação entre os agentes, hoje descentralizada, o que impede o acompanhamento sistemático; (iv) a padronização do registro

de pareceres, dado que um mesmo avaliador analisa simultaneamente trabalhos distintos em formatos heterogêneos; (v) a composição das bancas e o agendamento das defesas, que demandam conciliar a disponibilidade de múltiplos participantes; e (vi) a geração de relatórios gerenciais ao coordenador de TCC, ao NDE e ao colegiado, que agregam dados transversais aos trabalhos. Funcionalidades análogas constam em sistemas congêneres de apoio à gestão de TCC (Dorneles et al., 2023).

O processo administrativo vigente, resumido na Figura 1, apoia-se em etapas manuais e no SIGAA (matrícula em TCC-1 e TCC-2, entregas e lançamento de notas) (Universidade Federal de Itajubá, 2026). O sistema proposto posiciona-se como uma **camada de gestão complementar** a esse processo integrando comunicação, acompanhamento e avaliação, sendo a integração direta com o SIGAA considerada trabalho futuro.

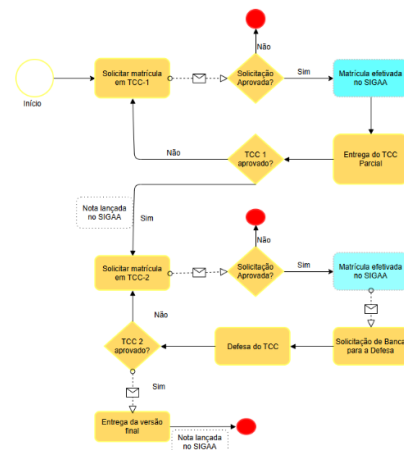


Figura 1: Fluxo administrativo atual da gestão de TCC (processo vigente, apoiado no SIGAA), apresentado como contexto. Fonte: (Universidade Federal de Itajubá, 2026).

Requisitos do sistema. A partir dessa análise, foram especificados os requisitos funcionais (Tabela 1) e não funcionais (Tabela 2). Os requisitos não funcionais foram definidos de modo a endereçar explicitamente as características críticas de uma solução distribuída *brokerless* em especial a persistência, a concorrência, a escalabilidade e a confiabilidade.

Componentes envolvidos. No espaço do problema, identificam-se cinco classes de atores, discente, orientador, coordenador de TCC, membros da banca avaliadora e órgãos colegiados (NDE e colegiado do curso) e os artefatos por eles manipulados documentos e suas versões, pareceres, bancas, defesas e relatórios. Esses elementos agrupam-se em domínios funcionais (documentos, notificações, bancas e defesas, relatórios e apoio por inteligência artificial) que, na etapa de modelagem, darão origem a serviços distribuídos autônomos, coordenados por **coreografia**, com canais de comunicação dedicados.

Justificativa da solução distribuída. Conforme fundamentado na Introdução e na Fundamentação Teórica, a autonomia dos agentes, a concorrência das interações e a assincronia entre elas tornam a abordagem distribuída adequada ao problema; os requisitos RNF01 a RNF04 traduzem diretamente essas características em critérios de projeto.

Cenários de uso. Três cenários sintetizam o funcionamento esperado e servem de base à modelagem (Subseção seguinte) e à avaliação do sistema:

Tabela 1: Requisitos funcionais do sistema.

ID	Requisito funcional
RF01	Autenticação de usuários e controle de perfis (discente, orientador, coordenador de TCC, banca avaliadora e órgãos colegiados).
RF02	Submissão e versionamento de documentos (proposta, versões parciais e versão final).
RF03	Acompanhamento do progresso e dos estados de cada TCC.
RF04	Registro padronizado de pareceres e revisões dos avaliadores.
RF05	Notificação de eventos aos agentes interessados.
RF06	Composição das bancas avaliadoras.
RF07	Agendamento das defesas.
RF08	Geração de relatórios gerenciais (ex.: defesas realizadas no prazo, distribuição por área de pesquisa e identificação de trabalhos com potencial de publicação).
RF09	Apoio por inteligência artificial ao acompanhamento: análise das versões, verificação do atendimento ao <i>feedback</i> e recomendação de correções básicas.

Tabela 2: Requisitos não funcionais do sistema.

ID	Requisito não funcional
RNF01	Comunicação assíncrona e orientada a eventos (coreografia) entre os processos distribuídos.
RNF02	Persistência e durabilidade do estado, mantendo uma fonte da verdade consultável.
RNF03	Suporte à concorrência, com orientações e avaliações ocorrendo simultaneamente.
RNF04	Escalabilidade por meio da segmentação da comunicação em canais por tipo de serviço.
RNF05	Confiabilidade na troca de mensagens, com identificadores e confirmações que assegurem idempotência.
RNF06	Interoperabilidade entre o <i>frontend</i> (JavaScript/React) e o <i>backend</i> (Python) por meio de JSON sobre HTTP.
RNF07	Utilização de tecnologias livres e sem restrição de uso.

1. **Submissão e revisão.** O discente submete uma nova versão do trabalho; o sistema a persiste e versiona; o serviço de IA analisa a versão e recomenda correções básicas; o orientador é notificado e registra um parecer padronizado, e o discente é notificado do retorno.
2. **Banca e defesa.** O orientador compõe a banca avaliadora e a encaminha ao coordenador de TCC, que agenda a defesa; os membros são notificados e confirmam a participação. Realizada a defesa e havendo aprovação, o discente corrige os pontos levantados pela banca e envia a versão final à coordenação, com cópia ao orientador, que endossa a entrega.
3. **Relatório gerencial.** O coordenador de TCC ou o NDE solicita um relatório (por exemplo, defesas no prazo ou distribuição por área de pesquisa); a agregação dos dados é distribuída a *workers* e o panorama consolidado é retornado ao solicitante.

3.2 Modelagem do Sistema

Esta subseção apresenta a arquitetura do sistema distribuído sob o paradigma de **coreografia**, seus componentes, o modelo de comunicação entre os processos e o fluxo de mensagens, apoiando-se em diagramas que evidenciam a interação entre os componentes.

Visão geral da arquitetura. A solução é organizada como um conjunto de serviços/agentes autônomos (*peers*) que cooperam por **coreografia**: cada serviço publica eventos e os interessados reagem diretamente a eles, **sem orquestrador nem broker central**. A realização no ZeroMQ adota uma **malha direta de publish-subscribe**: cada *peer* faz *bind* de um *socket* PUB para publicar seus eventos e *connect* de *sockets* SUB aos *peers* cujos eventos consome, sem um encaminhador central. A arquitetura distribui-se em três camadas: o *frontend* web, em React/JavaScript (Vite); o *backend* (a borda *Backend-for-Frontend* e os serviços), em Python com o *binding* PyZMQ; e a persistência, sob o MySQL, que atua como fonte da verdade e recupera o desacoplamento temporal ausente no transporte *brokerless*. A borda traduz as chamadas HTTP do *frontend* em eventos na malha, mas não faz parte da coreografia. As visões de contexto e de contêineres são apresentadas na Figura 2, segundo o modelo C4.

Componentes distribuídos. O sistema decompõe-se em oito serviços, derivados dos domínios de evento do fluxo (Tabela 3). Somam-se a eles o *frontend* web (React/JavaScript), a borda **BFF** (Python), o banco MySQL e o **Provedor de LLM** — uma dependência externa **plugável**, consumida pelo Serviço de IA. O protótipo implementa os oito serviços e exercita os três padrões de comunicação (PUB/SUB, DEALER/ROUTER e PUSH/PULL) nos três cenários (submissão e revisão; banca e defesa; relatórios gerenciais). A estrutura interna do Serviço de Documentos, protagonista desse cenário, é detalhada na Figura 3 (componentes e código).

Modelo de comunicação entre os processos. Os três padrões detalhados na Seção 2 distribuem-se entre os serviços conforme a Tabela 3: o *publish-subscribe* é a espinha dorsal da coreografia, com o tópico correspondendo ao tipo de evento, enquanto DEALER/ROUTER atende ao login síncrono e PUSH/PULL à geração distribuída de relatórios. Os serviços fazem *bind* de seus *endpoints* e os clientes fazem *connect*; as

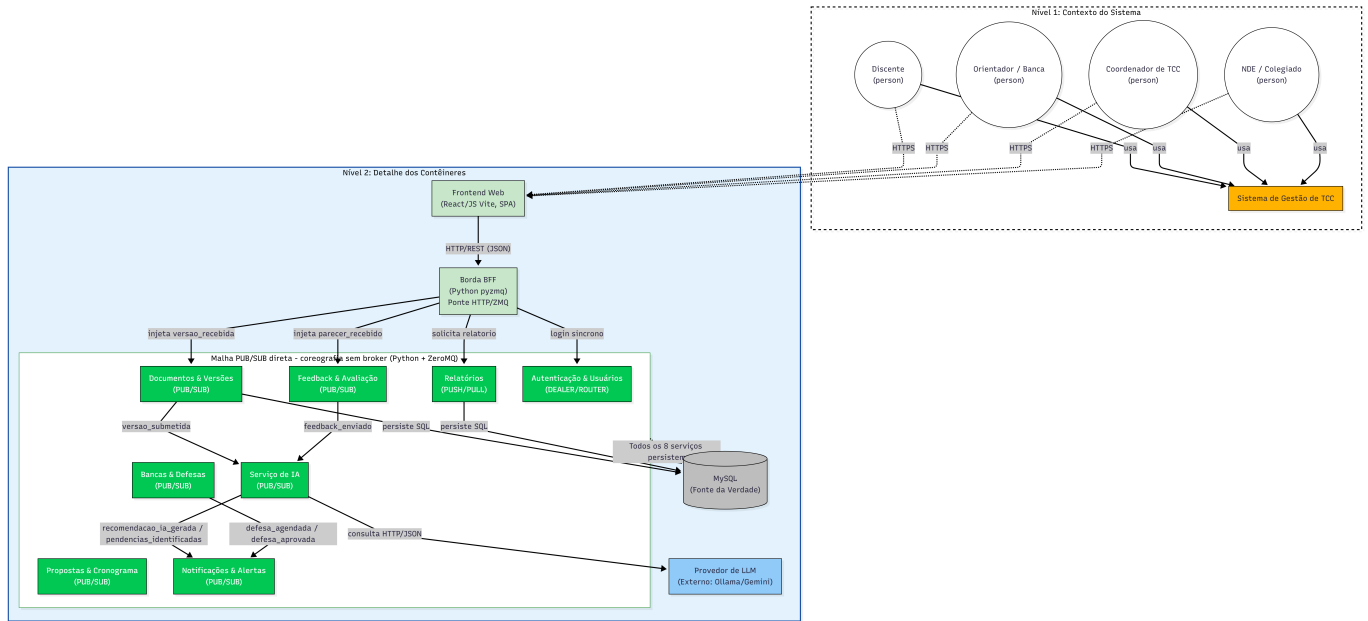


Figura 2: Modelo C4 do sistema: contexto (Nível 1) e contêineres (Nível 2) — *frontend* em React/JavaScript, *borda* (BFF) e serviços (*peers*) em Python (PyZMQ), MySQL e Provedor de LLM sobre a malha *publish-subscribe*.

Tabela 3: Serviços (*peers*) do sistema, responsabilidade e padrão de comunicação.

Serviço (<i>peer</i>)	Responsabilidade	Padrão
Autenticação & Usuários	Login e gestão de perfis de acesso.	DEALER/ROUTER
Propostas & Cronograma	Submissão e avaliação da proposta; registro do cronograma.	PUB/SUB
Documentos & Versões	Submissão e versionamento dos documentos do TCC.	PUB/SUB
Feedback & Avaliação	Feedbacks, notas (parcial e final) e decisões de aprovação.	PUB/SUB
Bancas & Defesas	Definição da banca, agendamento e convocação da defesa.	PUB/SUB
Notificações & Alertas	Alertas e comunicações (e-mails) aos agentes.	PUB/SUB
IA	Acompanhamento, avaliação de aptidão e análise da banca.	PUB/SUB
Relatórios	Geração distribuída de relatórios gerenciais.	PUSH/PULL

mensagens trafegam em JSON, segundo o esquema {evento, operação, identificador, *timestamp*, *payload*}. O **Serviço de IA** assina os eventos relevantes, monta um *prompt* a partir dos dados recebidos e consulta o Provedor de LLM por HTTP/JSON — sua **única chamada não-ZeroMQ** —, publicando em seguida os eventos de análise. Como o provedor é acessado por uma interface, sua troca (um modelo local ou um serviço em nuvem) não afeta a arquitetura (no protótipo, adotou-se a API do Google Gemini).

Fluxo de mensagens. O processo completo, da proposta à defesa e à geração de relatórios, é apresentado em notação BPMN na Figura 4. Para ilustrar a coreografia no nível das mensagens, detalha-se o cenário de submissão e revisão, cuja sequência consta na Figura 5. O discente submete uma nova versão pelo cliente, comando que a borda (BFF) injeta na malha como *versao_recebida* (do mesmo modo, *parecer_recebido*, *proposta_recebida* e *proposta_avaliada* nos demais fluxos); o Serviço de Documentos persiste e versiona a versão no MySQL e publica *versao_submetida*; o Ser-

viço de IA, inscrito nesse evento, monta um *prompt*, consulta o Provedor de LLM e publica *recomendacao_ia_gerada*. O Serviço de Feedback e Avaliação publica *feedback_enviado* a partir do parecer do orientador; o Serviço de IA reavalia e publica *feedback_atendido* ou *pendencias_identificadas*. Havendo pendências, o Serviço de Notificações publica *alerta_pendencia_disparado* ao orientador e, quando houver, ao coordenador. Nenhum agente central coordena a sequência: cada serviço reage diretamente aos eventos de seu interesse. Os demais cenários (banca e defesa; relatórios gerenciais) seguem o mesmo princípio.

Implantação. O caráter distribuído manifesta-se na implantação em múltiplos nós, interligados por conexões TCP do ZeroMQ entre os *peers* e por HTTP do Serviço de IA ao Provedor de LLM, conforme a Figura 6.

Formalização. Formalmente, o sistema é um conjunto de *peers* $P = \{p_1, \dots, p_n\}$ e um conjunto de tipos de evento (tópicos) E . Cada *peer* p_i define os eventos que publica, $\text{pub}(p_i) \subseteq E$, e os que assina, $\text{sub}(p_i) \subseteq E$. Há comunicação direta de p_i para p_j sempre que $\text{pub}(p_i) \cap \text{sub}(p_j) \neq \emptyset$, sem intermediário central. Cada mensagem é a tupla $m = (e, o, i, t, d)$, em que e é o evento (tópico), o a operação, i o identificador único, t o instante de envio e d o *payload* em JSON. O identificador i viabiliza a idempotência e a marca temporal t apoia a ordenação dos eventos. Vale observar que t é um *timestamp* físico; uma ordenação independente dos relógios das máquinas exigiria relógios lógicos: o de Lamport ordena os eventos de forma consistente com a relação *acontece-antes*, ao passo que os relógios vetoriais capturam a causalidade entre eventos. Esses mecanismos são considerados como evolução futura (Fridman, 2026c).

3.3 Implementação

A modelagem da subseção anterior foi materializada em um protótipo funcional que realiza a arquitetura proposta. O protótipo implementa os oito serviços (*peers*), a borda BFF e a in-

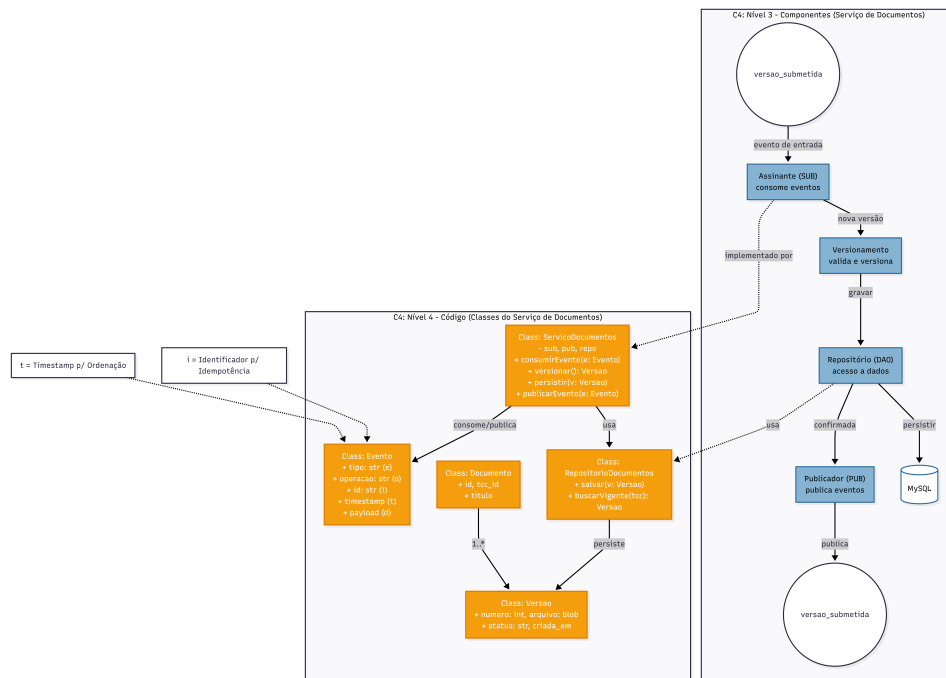


Figura 3: Serviço de Documentos: componentes (C4 Nível 3) e código (C4 Nível 4).

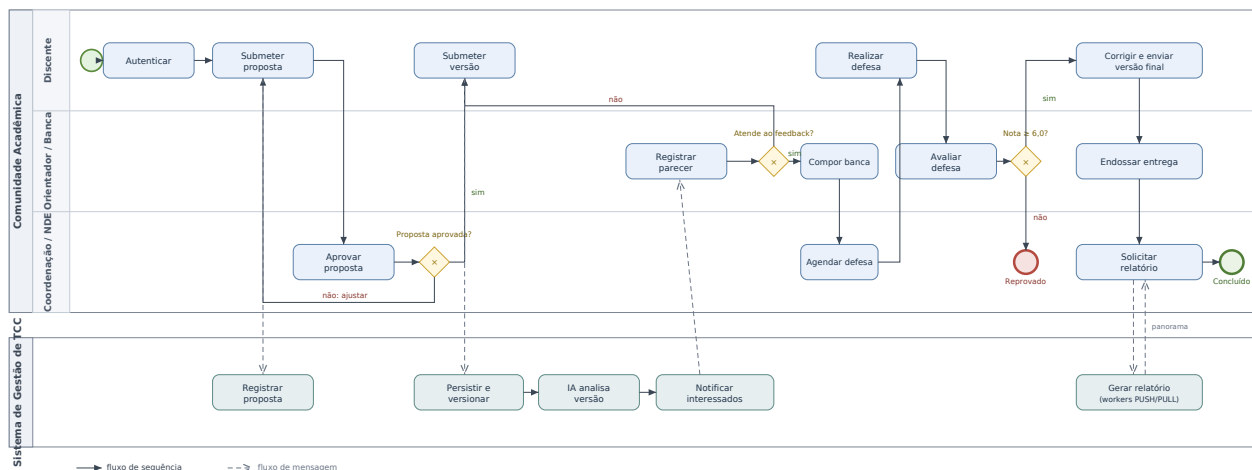


Figura 4: Processo completo de gestão de TCC em notação BPMN, da proposta à geração de relatórios.

terface web, exercita os três padrões de comunicação (*publish-subscribe*, DEALER/ROUTER e PUSH/PULL) e cobre os três cenários de uso. Por padrão, executa de forma autônoma e *offline*: o acesso a dados opera em modo de memória quando o MySQL não está presente, e o provedor de LLM opera em modo simulado, o que permite demonstrar a coreografia sem dependências externas; opcionalmente, conecta-se a um servidor MySQL e a um provedor de LLM real.

Organização do código. O repositório separa o *backend* distribuído, em Python, da interface, em React/JavaScript com *Vite*. Em backend/, o arquivo `run.py` inicia toda a malha e a borda em um único comando; `bff.py` é a borda BFF; o pacote `common/` concentra a configuração de portas, os tipos de evento, o *logger*, o objeto de acesso a dados (DAO) e o provedor de LLM; e `services/` reúne os oito serviços e os *workers* de relatório. O esquema do banco está em `services/database/mysql`, e a interface, em `src/`,

comunica-se com a borda por HTTP/JSON. O código-fonte completo do protótipo está disponível em <https://github.com/b-stanley/gestao-tcc-XRSC09>.

Borda BFF. A borda traduz cada chamada HTTP da interface em um evento injetado na malha: um *socket* PUB publica os comandos do cliente (`versao_recebida`, `parecer_recebido` e `banca_definida`, entre outros); um *socket* DEALER realiza o *login* síncrono junto ao serviço de Autenticação (ROUTER); e um *socket* PUSH solicita a geração de relatórios. Em sentido inverso, um *socket* SUB assina os eventos publicados pelos serviços e mantém um modelo de leitura (*read-model*) que a interface consulta por HTTP. Conforme a modelagem, a borda não participa da coreografia: ela apenas faz a ponte entre o *frontend* e a malha. As sessões usam *tokens* assinados por HMAC.

Serviços e padrões de comunicação. Cada serviço é um processo autônomo que assina os eventos de seu interesse

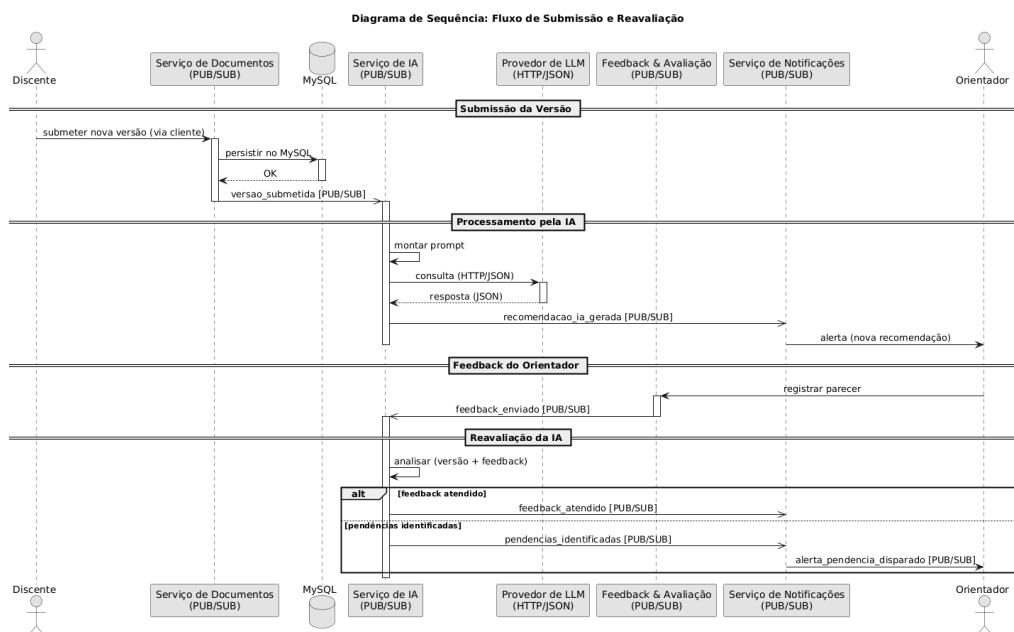
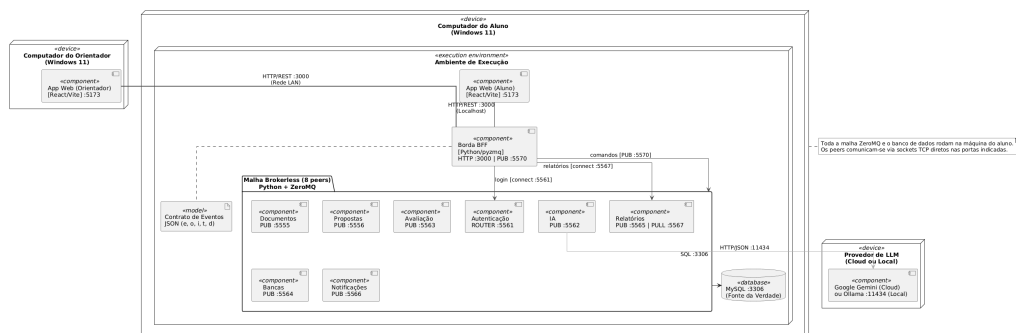


Figura 5: Diagrama de sequência do cenário de submissão e revisão (coreografia).



e publica os próprios, conforme o fluxo descrito na Subseção 3.2. No plano de implementação, o Serviço de Documentos reproduz as classes do C4 Nível 4 (`ServicoDocumentos`, `Documento`, `Versao` e `RepositorioDocumentos`); o Serviço de Feedback e Avaliação registra o parecer do orientador como um formulário padronizado (critérios, nota e decisão); o Serviço de Autenticação é o único síncrono, atendendo ao *login* pelo padrão DEALER/ROUTER; e o Serviço de Relatórios aplica o PUSH/PULL, distribuindo a agregação a *workers* em paralelo e consolidando os parciais. O Código 1 ilustra o padrão de um *peer*: assina o evento de entrada, processa de forma idempotente e publica o evento de saída.

Código 1: Serviço de Documentos: reage a *versao_recebida*, persiste e publica *versao_submetida*.

```

1 sub = ctx.socket(zmq.SUB); sub.connect(
    get_zmq_address("gateway"))
2 sub.setsockopt_string(zmq.SUBSCRIBE, "
    versao_recebida")
3 pub = ctx.socket(zmq.PUB); pub.bind(
    get_zmq_address("documentos","bind"))
4 while True:
5     _, corpo = sub.recv_string().split(" ",
        1)
6     dados = json.loads(corpo)

```

```

7      if not repo.evento_novo(dados):          #
            idempotencia (RNF05)
8      continue
9      numero = repo.salvar-versao(dados["
            aluno_id"], dados["payload"]["texto
            "])
10     ev = Evento(TipoEvento.VERSAO_SUBMETIDA
            , aluno_id=dados["aluno_id"],
            payload={ "numero": numero})
11     pub.send_string(f"{ev.evento} {ev.
12         to_json_str()}")

```

Mensagens e idempotência. Toda mensagem é uma instância da classe `Evento`, serializada em JSON no formato $m = (e, o, i, t, d)$ definido na modelagem. Antes de processar um evento, cada serviço registra seu identificador i na trilha de eventos e ignora repetições, o que assegura a idempotência exigida pelo RNF05 mesmo diante de reenvios.

Persistência. O MySQL é a fonte da verdade, com tabelas para versões, pareceres, bancas, defesas, notificações e a trilha de eventos (`historico_eventos`). Para que o protótipo seja executável sem um servidor de banco, o DAO oferece um modo de memória equivalente; quando o MySQL está presente, a borda reidrata seu modelo de leitura a partir dele ao iniciar, recuperando o desacoplamento temporal ausente no

transporte *brokerless* (RNF02).

Provedor de LLM plugável. O acesso ao modelo de linguagem é abstraído pela interface `ProvedorLLM` (C4 Nível 4), com implementações para o modo simulado (heurística local), para um modelo local via Ollama, para um *stub* HTTP e para a API do Google Gemini. A seleção ocorre por variável de ambiente: na execução e na demonstração do protótipo, adotou-se a **API do Google Gemini** (modelo `gemin-3.5-flash`) como provedor; o modo simulado permanece como alternativa *offline* e como retorno automático em caso de falha, de forma que a coreografia não é interrompida. Essa consulta é a única comunicação do sistema que não usa ZeroMQ.

Endpoints e execução. A interface é servida pelo *Vite* (porta 5173, com *proxy* de `/api` para a borda) e a borda expõe a API HTTP na porta 3000; cada *peer* publica em uma porta TCP própria, o que concretiza a malha direta sem intermediário. Os *endpoints* e as portas usados na execução e no teste são apresentados no diagrama de implantação (Figura 6).

4 Avaliação e Resultados

A avaliação do protótipo seguiu duas frentes complementares: a demonstração funcional dos três cenários de uso, que verifica se a coreografia produz o comportamento esperado, e a medição quantitativa da cadeia de submissão e revisão, que examina o desempenho da comunicação. As medições foram obtidas em uma única máquina, com os *peers* comunicando-se por ZeroMQ sobre TCP em *loopback*, no modo *offline* (acesso a dados em memória e provedor de IA simulado), de forma a garantir reprodutibilidade e isolar o custo do transporte.

Evidência funcional da coreografia. No cenário de submissão e revisão, um único comando do cliente (`versao_recebida`), injetado pela borda, dispara, sem qualquer coordenador central, a cadeia autônoma de reações: o Serviço de Documentos persiste e versiona o documento e publica `versao_submetida`; o Serviço de IA, inscrito nesse evento, monta o *prompt*, consulta o provedor e publica `recomendacao_ia_gerada`; o Serviço de Notificações registra o aviso e, havendo pendências na reavaliação, publica `alerta_pendencia_disparado`. Cada mensagem trafega como a tupla $m = (e, o, i, t, d)$ e é descartada quando seu identificador i já foi processado, o que confirma a idempotência. A observação dos eventos publicados, e não da interface, evidencia que cada serviço reage apenas aos eventos de seu interesse, caracterizando a coreografia sem orquestrador. O Código 2 apresenta um trecho do registro de execução dessa cadeia, capturado durante a execução do protótipo, com os identificadores de cada evento e os instantes de cada etapa.

Código 2: Registro de execução da coreografia (cenário de submissão e revisão), com identificadores de evento.

```

1 [Cliente/BFF]      injeta versao_recebida
                      (id dd62741c)
2 [Documentos]      publica versao_submetida
                      (id 2afb12f5) +2,3 ms
3 [IA]               LLM (score 30) ->
                      recomendacao_ia_gerada (id 85684116)
                      +0,6 ms
4 [Notificacoes]    notifica o orientador
5 [Orientador/BFF]  injeta parecer_recebido
                      (id e8d7059a)
```

```

6 [Feedback&Aval]   publica feedback_enviado
                      (id 4a1a9382)
7 [IA]              reavalua ->
                      pendencias_identificadas (id 2273241f)
8 [Notificacoes]    publica
                      alerta_pendencia_disparado ->
                      orientador (id d99b164d)
```

Medições quantitativas. Para sustentar empiricamente as propriedades de baixa latência e vazão, instrumentou-se a cadeia de submissão e revisão sobre o mesmo transporte (ZeroMQ *publish-subscribe*), reutilizando o formato de mensagem e o provedor de IA do sistema. A Tabela 4 resume os resultados. Sob carga leve (cerca de cem submissões por segundo), a latência ponta-a-ponta da cadeia completa ficou em torno de 2 ms, com saltos individuais abaixo de 1 ms, o que é coerente com o desempenho esperado do ZeroMQ frente ao uso direto de *sockets* (Treat, 2014). Em uma rajada de três mil eventos, a malha sustentou aproximadamente 4500 eventos por segundo sem qualquer perda; sob essa saturação, a latência ponta-a-ponta subiu para a faixa de 300 ms, efeito de enfileiramento condizente com a entrega de melhor esforço do *publish-subscribe* (Treat, 2014; ZeroMQ, 2026b). A análise pelo provedor simulado mostrou-se desprezível (cerca de 0,01 ms); um provedor real, local ou em nuvem, acrescentaria latência de rede e de inferência, o que reforça a decisão de isolar essa consulta como a única chamada não-ZeroMQ do sistema, executada de forma assíncrona para não bloquear a malha.

Tabela 4: Medições do cenário de submissão e revisão (uma máquina, ZeroMQ em *loopback*, provedor de IA simulado).

Métrica	Valor
Latência ponta-a-ponta, carga leve (média / mediana / p95)	2,0 / 1,9 / 3,4 ms
Salto Documentos (<code>versao_recebida</code> → <code>versao_submetida</code>)	0,9 ms
Salto IA (inclui a análise, → <code>recomendacao_ia_gerada</code>)	0,6 ms
Salto Notificações (entrega do evento)	0,5 ms
Análise da IA (provedor simulado)	0,01 ms
Vazão sob rajada de 3000 eventos	≈ 4500 eventos/s
Eventos recebidos / enviados (burst)	3000 / 3000 (sem perda)
Latência ponta-a-ponta sob rajada (média / p95)	318 / 552 ms

Atendimento aos requisitos não funcionais. Os resultados oferecem evidências para os requisitos definidos na análise do problema. A cadeia autônoma de eventos confirma a comunicação assíncrona e orientada a eventos (RNF01). A vazão sustentada sem perda, com submissões processadas de forma concorrente, sustenta a concorrência (RNF03) e a escalabilidade obtida pela segmentação da comunicação em um canal por serviço (RNF04). O descarte de eventos repetidos por identificador e a ausência de perdas no teste apoiam a confiabilidade por idempotência (RNF05). A persistência e a fonte da verdade (RNF02) são providas pelo MySQL, com modo de memória de contingência e reidratação do estado pela borda. A interoperabilidade entre o *frontend* e o *backend* por JSON

sobre HTTP (RNF06) e o uso exclusivo de tecnologias livres (RNF07) decorrem diretamente das escolhas de implementação.

Limitações da avaliação. As medições foram conduzidas em uma única máquina e em *loopback*, de modo que os tempos representam o custo do transporte e do processamento, e não o de uma rede real; a implantação de teste em duas máquinas exercita as interfaces em rede, mas não distribui a malha. A entrega é de melhor esforço, sem semântica *exactly-once*, e a ordenação apoia-se em marca temporal física, e não em relógios lógicos. O provedor de IA utilizado nas medições é o simulado, escolhido pela reprodutibilidade, embora o protótipo opere, na demonstração funcional, com a API do Google Gemini (modelo *gemini-3.5-flash*); a avaliação quantitativa de um provedor real, com medição de latência de rede e de inferência, é considerada como trabalho futuro. Essas limitações não invalidam os resultados, mas delimitam seu alcance e indicam caminhos de aprofundamento.

5 Conclusão

Este trabalho projetou e implementou um protótipo funcional de um sistema distribuído para a gestão de TCC, aplicando o ZeroMQ sob o paradigma *brokerless* com coordenação por coreografia. O problema de origem, uma gestão fragmentada, com comunicação descentralizada, controle de versões disperso, pareceres sem padronização e ausência de visão gerencial, é endereçado por uma malha de oito serviços autônomos, um por domínio do processo, que reagem a eventos em JSON sem *broker* nem orquestrador central. A segmentação da comunicação em um canal por serviço integra interações antes dispersas; o MySQL provê a fonte da verdade que o transporte *brokerless* não persiste; e o serviço de inteligência artificial apoia o acompanhamento, reduzindo o esforço com correções básicas recorrentes. A avaliação confirmou a viabilidade técnica: a cadeia de eventos opera de ponta a ponta com latência da ordem de milissegundos e sustenta carga sem perdas.

A solução posiciona-se como uma camada de gestão complementar ao processo administrativo vigente. Sistemas institucionais consolidados, como o SIGAA, poderiam estudar as possibilidades de integração com uma plataforma desse tipo, delegando a ela a comunicação, o acompanhamento e a avaliação apoiada por inteligência artificial, enquanto mantêm o registro acadêmico oficial. Essa integração, não realizada aqui, é um desdobramento natural do trabalho.

O caráter *brokerless* trouxe descentralização e baixa latência ao custo de garantias de entrega e de persistência nativas, recuperadas na camada de aplicação por idempotência e estado durável. Entre as dificuldades enfrentadas no desenvolvimento, destacaram-se a incompatibilidade do *binding* nativo de ZeroMQ para Node.js no ambiente de teste, que motivou a reescrita da borda em Python, e a obtenção de confiabilidade sobre um transporte sem garantias nativas de entrega. Esse compromisso, assumido de forma explícita, aponta os principais caminhos de evolução: entrega confiável por um protocolo como o *Majordomo*, ordenação causal por relógios lógicos, a distribuição da malha em mais nós e uma avaliação quantitativa com provedor de LLM real sobre uma rede física. Ao reunir comunicação, tempo, coordenação e transações em

uma solução coerente, o trabalho demonstra, na prática, a aplicação dos conceitos centrais de sistemas distribuídos ao domínio da gestão de TCC.

Contribuições dos autores

Bruno foi responsável pela arquitetura e pelo desenvolvimento do sistema; Larissa, pela coordenação do projeto, análise do problema e pela avaliação experimental e validação; Patricki, pela modelagem do sistema; e Patricky, pelo apoio ao relatório.

Referências

- Dorneles, R. C., Castanho, C. L. d. O., Cavalheiro, B. O., & Minuzzi, P. D. (2023). Sistema integrado para gerenciamento de trabalhos de conclusão de curso. Universidade Regional Integrada do Alto Uruguai e das Missões (URI), Santiago, RS, Brasil. Acesso em: 8 jun. 2026.
- Frinhani, R. (2026a). Comunicação distribuída — notas de aula da disciplina *xrsc09 sistemas distribuídos*. Material didático.
- Frinhani, R. (2026b). Coordenação em sistemas distribuídos — notas de aula da disciplina *xrsc09 sistemas distribuídos*. Material didático.
- Frinhani, R. (2026c). Tempos em sistemas distribuídos — notas de aula da disciplina *xrsc09 sistemas distribuídos*. Material didático.
- Frinhani, R. (2026d). Transação distribuída — notas de aula da disciplina *xrsc09 sistemas distribuídos*. Material didático.
- Hintjens, P. (2013). *ZeroMQ: Messaging for Many Applications*. Sebastopol, CA: O'Reilly Media. Disponível também como “The ZeroMQ Guide”: <https://zguide.zeromq.org/>.
- Lauener, J. & Sliwinski, W. (2017). How to design & implement a modern communication middleware based on ZeroMQ. Em *Proceedings of the 16th International Conference on Accelerator and Large Experimental Physics Control Systems (ICALPCS'17)* Barcelona, Spain: JACoW Publishing.
- Sociedade Brasileira de Computação (2026). JEMS — journal and event management system. <https://jems.sbc.org.br/>. Mantido pela UFRGS; adaptação do sistema EDAS. Acesso em: 8 jun. 2026.
- Treat, T. (2014). Distributed messaging with ZeroMQ. Brave New Geek. <https://bravenewgeek.com/distributed-messaging-with-zeromq/>. Acesso em: 28 maio 2026.
- Universidade Federal de Itajubá (2026). Trabalho de conclusão de curso — sistemas de informação: matrícula (grade nova 2022). <https://sites.google.com/unifei.edu.br/tcc-sin/matr%C3%ADcula/grade-nova-2022>. Acesso em: 8 jun. 2026.
- ZeroMQ (2026a). ZeroMQ — get started. <https://zeromq.org/get-started/>. Acesso em: 28 maio 2026.
- ZeroMQ (2026b). ZeroMQ — socket API. <https://zeromq.org/socket-api/>. Acesso em: 28 maio 2026.

